

CODOMAX DIGITAL SOLUTIONS

INTERNSHIP PROJECT REPORT

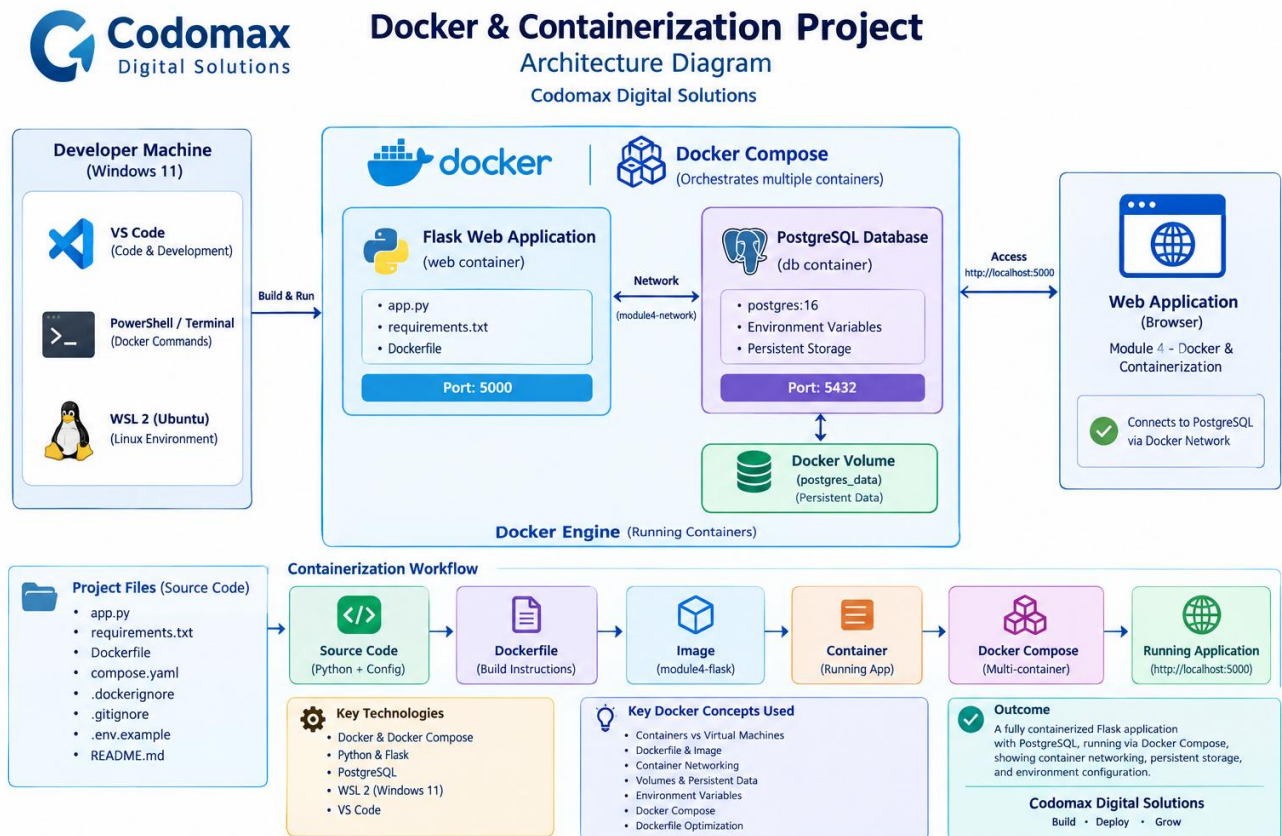
MODULE 4

Docker & Containerization

Submitted By: Sikandar Shah

Organization: Codomax Digital Solution

Module 04: Docker & Containerization



Prepared for internship submission

1. Executive Summary

This report documents the Module 4 practical project, “Docker & Containerization,” completed as part of the internship at Codomax Digital Solutions. The module introduces the fundamentals of containerization and demonstrates how a Python Flask web application can be packaged, built, deployed, and connected to a PostgreSQL database using Docker and Docker Compose.

The project follows a practical workflow: Source Code → Dockerfile → Docker Image → Docker Container → Docker Compose → Running Application. It also demonstrates container networking, persistent storage through Docker volumes, environment variables, service communication, and basic Dockerfile optimization.

2. Module Objectives

- Understand containers and how they differ from traditional virtual machines.
- Understand Docker architecture, images, containers, and Dockerfiles.
- Practice essential Docker commands for building, running, inspecting, stopping, and removing containers.
- Understand container networking and service-to-service communication.
- Use Docker volumes for persistent database storage.
- Use environment variables for application and database configuration.
- Use Docker Compose to orchestrate a Flask web service and PostgreSQL database.
- Containerize a working web application and verify it through a browser.
- Demonstrate PostgreSQL connectivity from the Flask container.
- Apply basic Dockerfile optimization techniques.

3. Tools & Technologies

Technology / Tool	Purpose
Docker	Container platform used to build and run the application.
Docker Compose	Defines and runs the multi-container application stack.
Python / Flask	Web application framework and application source code.
PostgreSQL	Relational database service used by the application.
Dockerfile	Instructions for building the Flask application image.
Docker Network	Allows application services to communicate by service name.
Docker Volume	Provides persistent storage for PostgreSQL data.
WSL 2 / Ubuntu	Linux environment integrated with Windows for Docker development.
PowerShell / Terminal	Used to execute Docker and project commands.

4. System Architecture

The architecture contains a developer environment, a Docker Engine, a Flask web container, a PostgreSQL database container, a Docker network, a persistent volume, and a browser-based application interface.

- **Developer Machine:** Windows 11 with VS Code, PowerShell/Terminal, and WSL 2 (Ubuntu).
- **Docker Engine:** Runs the application and database containers.
- **Flask Web Container:** Hosts the Python Flask application on port 5000.
- **PostgreSQL Container:** Runs PostgreSQL 16 and listens on its internal database port 5432.
- **Docker Network:** Allows the Flask service to reach PostgreSQL using the database service name.
- **Docker Volume:** Stores PostgreSQL data persistently outside the temporary container layer.
- **Browser:** Accesses the Flask application through `http://localhost:5000`.

5. Containerization Workflow

- 1. Source Code:** `app.py` and `requirements.txt` define the Flask application and its dependencies.
- 2. Dockerfile:** Contains the instructions required to build the application image.
- 3. Docker Image:** The Dockerfile is built into a reusable image such as `module4-flask`.
- 4. Docker Container:** The image is started as a running Flask application container.
- 5. Docker Compose:** Coordinates the Flask and PostgreSQL services as one application stack.
- 6. Running Application:** The final application is available from the browser and can test database connectivity.

6. Project Structure

```
module-4-docker/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
├── compose.yaml  
├── .dockerignore  
├── .gitignore  
├── .env.example  
└── README.md
```

7. Implementation Details

7.1 Docker Environment Verification

The environment was prepared using WSL 2 and Docker. Docker and Docker Compose versions were checked from PowerShell, followed by the standard Docker hello-world image to verify that Docker could successfully pull and run a container.

Key verification commands:

```
wsl -l -v  
  
docker --version  
  
docker compose version  
  
docker info  
  
docker run hello-world
```

7.2 Flask Application

A lightweight Flask application was created as the project web service. The application listens on 0.0.0.0:5000 so that the service can be reached from the Docker host through the published port.

7.3 Dockerfile

The Dockerfile uses a slim Python base image, establishes /app as the working directory, copies dependency information before application source code, installs dependencies without retaining pip's package cache, exposes port 5000, and starts the Flask application.

```
FROM python:3.12-slim  
  
WORKDIR /app  
  
COPY requirements.txt .  
  
RUN pip install --no-cache-dir -r requirements.txt  
  
COPY . .  
  
EXPOSE 5000  
  
CMD ["python", "app.py"]
```

7.4 Docker Image & Container

The application image is built with the tag `module4-flask`. A container named `module4-web` is then created with port 5000 published from the container to the host.

```
docker build -t module4-flask .
```

```
docker images
```

```
docker run -d --name module4-web -p 5000:5000 module4-flask
```

```
docker ps
```

7.5 Docker Networking

A Docker network provides an isolated communication environment for containers. In the Compose application, the Flask service communicates with PostgreSQL using the database service name rather than relying on a host-machine database address.

Network example: `docker network create module4-network`

7.6 Persistent Storage with Docker Volumes

PostgreSQL data is stored in a named Docker volume. This separates persistent database data from the temporary lifecycle of a database container. The volume can remain available when containers are stopped and recreated, unless the volume is explicitly removed.

Volume example: `docker volume create module4-postgres-data`

7.7 Environment Variables

Database configuration is supplied through environment variables such as `DB_HOST`, `DB_PORT`, `POSTGRES_DB`, `POSTGRES_USER`, and `POSTGRES_PASSWORD`. The project includes an `.env.example` template so that configuration can be documented without publishing real secrets.

7.8 Docker Compose

Docker Compose defines the web and database services in a single `compose.yaml` file. The web service is built from the local `Dockerfile`, while the database uses the PostgreSQL 16 image. Compose also connects the services and attaches persistent storage to PostgreSQL.

```
services:
  web:
    build: .
    ports:
      - "5000:5000"
    environment:
      DB_HOST: db
      DB_PORT: 5432
      POSTGRES_DB: module4db
      POSTGRES_USER: module4user
      POSTGRES_PASSWORD: change-this-password
    depends_on:
      - db

  db:
    image: postgres:16
    environment:
      POSTGRES_DB: module4db
      POSTGRES_USER: module4user
      POSTGRES_PASSWORD: change-this-password
    volumes:
      - postgres_data:/var/lib/postgresql/data

volumes:
  postgres_data:
```

7.9 Database Connectivity Test

The Flask application includes a `/db-test` endpoint. It reads the database connection configuration from environment variables and attempts to connect to PostgreSQL through the Docker network. A successful response confirms service-to-service communication.

Expected browser result: Database connection successful!

7.10 Dockerfile Optimization

- Use a slim Python base image to reduce unnecessary image size.
- Copy requirements.txt before application source to improve Docker build-cache reuse.
- Use pip install --no-cache-dir to avoid storing pip's package cache in the image.
- Use .dockerignore to exclude .venv, __pycache__, .git, .env, and unnecessary files from the build context.
- Keep dependencies limited to what the application actually needs.

8. Testing & Verification

Test Area	Verification
Docker installation	Docker version and Docker Compose version commands return successfully.
Docker runtime	docker run hello-world confirms the Docker engine can run a container.
Image build	module4-flask appears in docker images after a successful build.
Container runtime	module4-web appears in docker ps with port 5000 published.
Web application	The Flask application is accessible at http://localhost:5000.
Network	Docker network is created and used for service communication.
Volume	PostgreSQL uses persistent named storage.
Compose	Flask and PostgreSQL services are started together with Docker Compose.
Database connection	/db-test confirms Flask can communicate with PostgreSQL.

9. Evidence & Screenshots

Figure 1: WSL 2 Verification

```
PS C:\Users\HB Comp> wsl -l -v
  NAME      STATE      VERSION
* Ubuntu    Running    2
PS C:\Users\HB Comp> |
```

Figure 2: Docker & Compose Version Verification

```
PS C:\Users\HB Comp> docker --version
Docker version 29.8.0, build 88096ef
PS C:\Users\HB Comp> docker compose version
Docker Compose version v5.5.1
PS C:\Users\HB Comp>
```

Figure 3: Docker Hello World

```
PS C:\Users\HB Comp> docker run hello-world

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

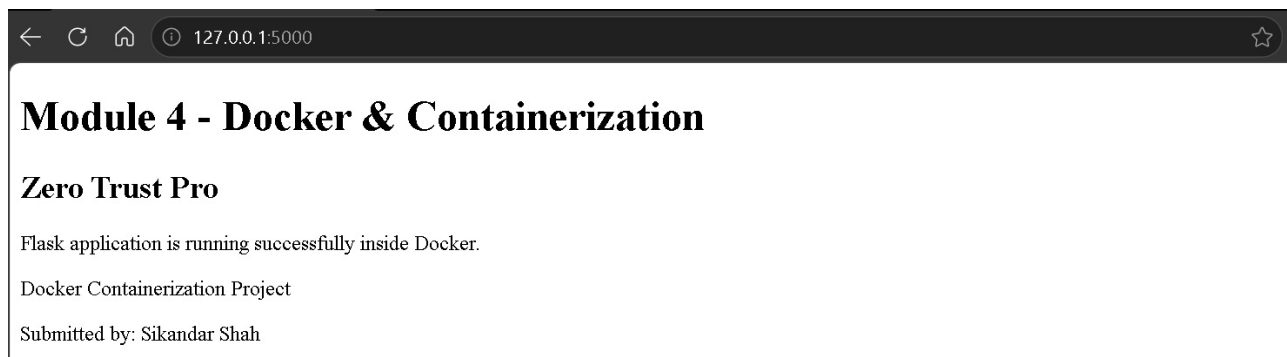
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

PS C:\Users\HB Comp>
```

Figure 4: Flask Application Running in Docker



The screenshot shows a web browser window with the address bar displaying '127.0.0.1:5000'. The page content includes the title 'Module 4 - Docker & Containerization', a sub-header 'Zero Trust Pro', and a message stating 'Flask application is running successfully inside Docker.' Below this, it says 'Docker Containerization Project' and 'Submitted by: Sikandar Shah'.

Figure 5: Docker Image

```
PS C:\Users\HB Comp\Documents\module-4-docker> docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
hello-world:latest	5e2309035332	25.9kB	9.49kB	U
module4-flask:latest	24bed93ae5df	198MB	48.2MB	

```
PS C:\Users\HB Comp\Documents\module-4-docker>
```

Figure 6: Running Container

```
PS C:\Users\HB Comp\Documents\module-4-docker> docker run -d --name module4-web -p 5000:5000 module4-flask
96fd3a436820e2678427d41680cc9c32d180c79a143ee69afd54a5c3a1dbdd54
PS C:\Users\HB Comp\Documents\module-4-docker> docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS
96fd3a436820   module4-flask  "python app.py"         21 seconds ago Up 20 seconds  0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
module4-web
PS C:\Users\HB Comp\Documents\module-4-docker>
```

Figure 7: Docker Network

```
PS C:\Users\HB Comp\Documents\module-4-docker> docker network create module4-network
ecbcfa78817f01b921ab735e32d69a3e20299925a5e786576d6628020e5622f0
PS C:\Users\HB Comp\Documents\module-4-docker> docker network ls
NETWORK ID     NAME                DRIVER              SCOPE
9b2c239967ba   bridge              bridge              local
d054e2cc0f5e   host                host                local
ecbcfa78817f   module4-network     bridge              local
17e6d9c35067   none                null                local
PS C:\Users\HB Comp\Documents\module-4-docker>
```

Figure 8: Docker Volume

```
PS C:\Users\HB Comp\Documents\module-4-docker> docker volume ls
DRIVER      VOLUME NAME
local       module4-postgres-data
PS C:\Users\HB Comp\Documents\module-4-docker>
```

Figure 9: Docker Compose

```
PS C:\Users\HB Comp\Documents\module-4-docker> docker compose up --build
web-1 | * Running on all addresses (0.0.0.0)
web-1 | * Running on http://127.0.0.1:5000
web-1 | * Running on http://172.19.0.3:5000
web-1 | Press CTRL+C to quit
db-1 | performing post-bootstrap initialization ... ok
db-1 | syncing data to disk ... ok
db-1 |
db-1 | Success. You can now start the database server using:
db-1 |
db-1 |     pg_ctl -D /var/lib/postgresql/data -l logfile start
db-1 |
db-1 | initdb: warning: enabling "trust" authentication for local connections
db-1 | initdb: hint: You can change this by editing pg_hba.conf or using the option -A, or --auth-local and --auth-host the next time you run initdb.
```

Figure 10: Database Connectivity

localhost:5000/db-test

Database connection successful!

10. Learning Outcomes

- Understood the purpose of containerization and the relationship between images and containers.
- Learned the basic Docker command-line workflow.
- Learned how a Dockerfile converts application requirements into a reproducible image.
- Learned how ports expose containerized services to the host.
- Understood container networking and service-name based communication.
- Understood why database data requires persistent storage.
- Learned how Docker Compose manages multiple related services.
- Practiced environment-based configuration.
- Applied basic Dockerfile optimization and build-cache concepts.
- Gained practical experience containerizing a Flask application with PostgreSQL.

11. Challenges & Solutions

Challenge	Approach / Solution
Understanding Docker concepts	Broke the workflow into image, container, network, volume, and Compose concepts and verified each step independently.
Container-to-container communication	Used Docker Compose networking and the database service name as the database host.
Persistent database storage	Attached PostgreSQL to a named Docker volume.
Efficient image builds	Applied a slim base image, .dockerignore, dependency-first COPY order, and --no-cache-dir.
Safe configuration	Used .env.example and excluded .env from version control.

12. Submission & Repository

After the practical work is completed, the project can be uploaded to a GitHub repository containing the source code, Docker configuration files, README documentation, and selected evidence screenshots. The GitHub repository link and LinkedIn post link can then be submitted as the module deliverables.

Recommended repository structure:

- Source code: app.py
- Dependencies: requirements.txt
- Container build: Dockerfile
- Multi-container orchestration: compose.yaml
- Configuration templates: .env.example
- Documentation: README.md

13. Conclusion

Module 4 provided practical exposure to modern containerization using Docker. The completed workflow demonstrates how a Flask application can be packaged into a Docker image, executed as a container, connected to PostgreSQL through a Docker network, and managed together with Docker Compose. Persistent storage, environment variables, verification procedures, and Dockerfile optimization were also incorporated. The project establishes a practical foundation for progressing toward more advanced container deployment, cloud infrastructure, and DevOps workflows.

Codomax Digital Solutions

Module 4 — Docker & Containerization